AUTONOMUS AGENTS AND MULTI-AGENT SYSTEMS
MULTI-AGENT RL

Lukas Esterle | lukas.esterle@ece.au.dk

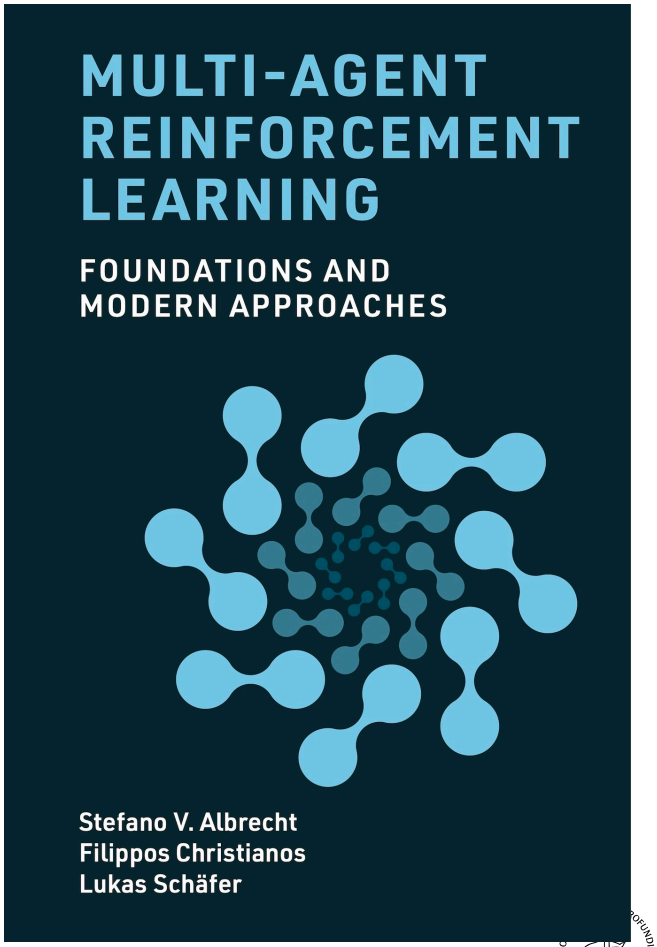
Mirgita Frasheri | mirgita.frasheri@ece.au.dk

SOURCE

Stefano V. Albrecht, Filippos Christianos, and Lukas Schäfer.

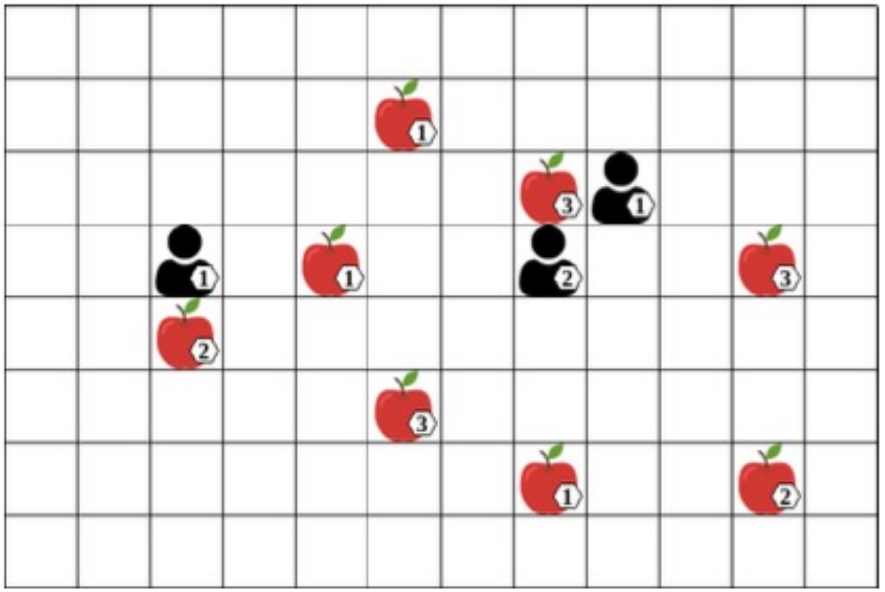
Multi-Agent Reinforcement Learning: Foundations and Modern Approaches.

MIT Press, 2024.



MOTIVATION FOR RL

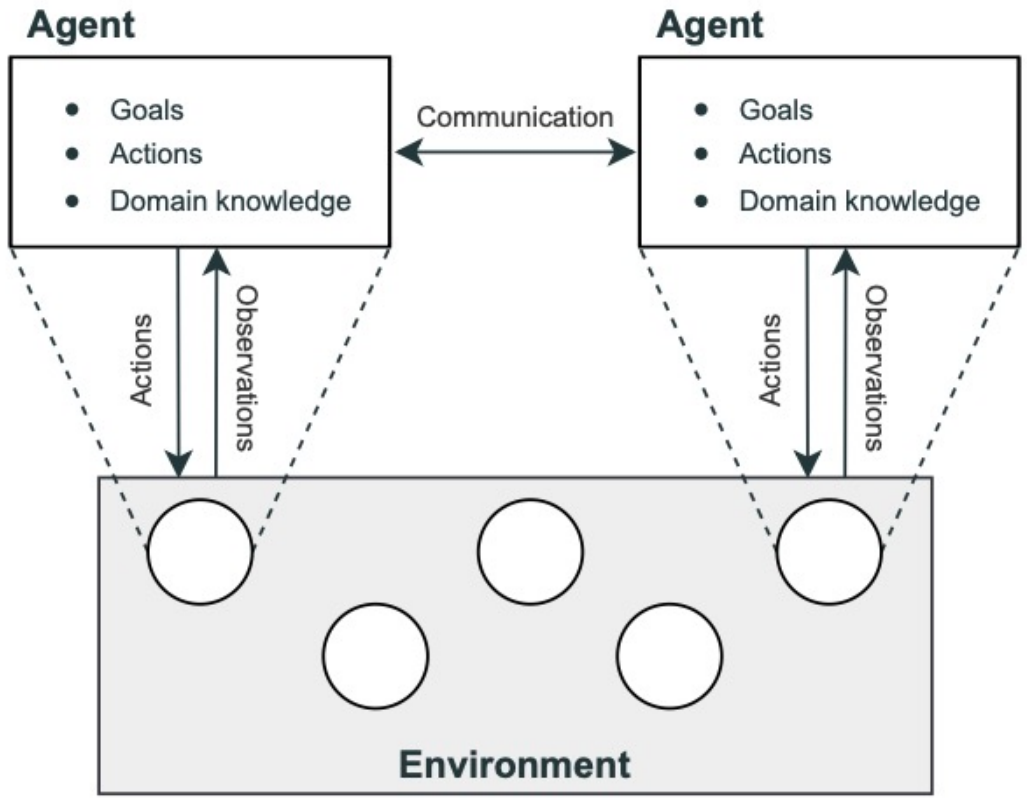
- Three (or more) agents (robots) with varying skill levels
- Goal: to **collect all items** (apples)
- Items can be collected if a group of one or more agents are located **next to the item and the sum of agents' levels is greater than or equal to the item level**
  (this is called Level Based Foraging (LBF))



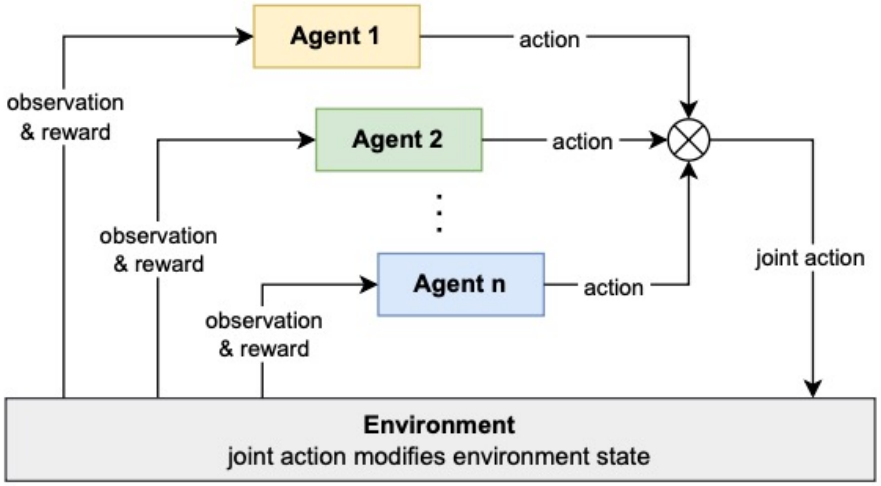
- Action space

  $A = \{up, down, left, right, collect, noop\}$

MULTI-AGENT SYSTEMS



MARL FOR SOLVING MULTI-AGENT SYSTEMS



- **Goal**: learn optimal policies for a set of agents in a multi-agent system
- Each agent chooses an action based on its policy ⇒ joint action
- **Joint action affects environment state;** agents get rewards + new observations

WHY DO WE NEED MARL

Why should we use MARL to find optimal solutions to multi-agent systems rather than controlling multiple 'agents' using a single-agent RL (SARL) algorithm?

Decomposing a large problem

- In LBF example, controlling 3 robots each with 6 actions, the joint action space becomes $6^3 = 216$.
  ⇒ Large action space for SARL!

- We can decompose this into three independent agents, each selecting from only 6 actions.
  ⇒ Use MARL to train separate agent policies (more tractable)

Decentralized decision making

- There are many real-world scenarios where it is required for each agent to make decisions independently.

- E.g. autonomous driving is impractical for frequent long-distance data exchanges between a central agent and the vehicle.

CHALLENGES OF MARL

New challenges arise in MARL:

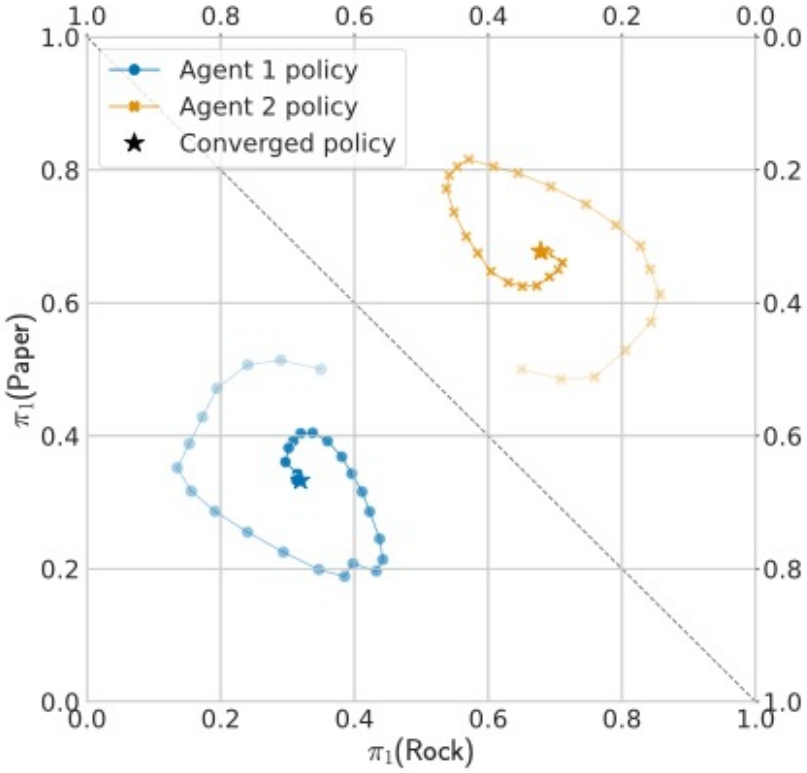
- Non-stationarity caused by multiple learning agents
- Optimality of policies and equilibrium selection
- Multi-agent credit assignment
- Scaling in number of agents

CHALLENGES OF MULTI-AGENT LEARNING

Non-stationary environment:

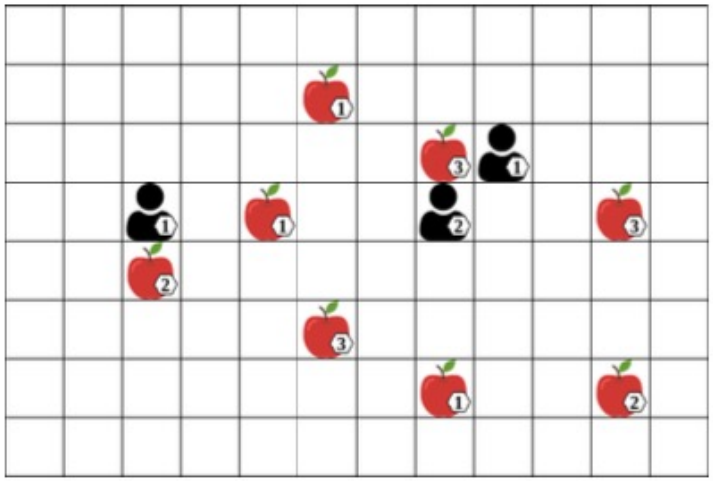
If multiple agents are learning, the environment becomes non-stationary from the perspective of individual agents

⇒ **Moving target**: each agent is optimizing against changing policies of other agents



CHALLENGES IN MARL – CREDIT ASSIGNMENT

Multi-agent credit assignment: which agent's actions contributed to received rewards?

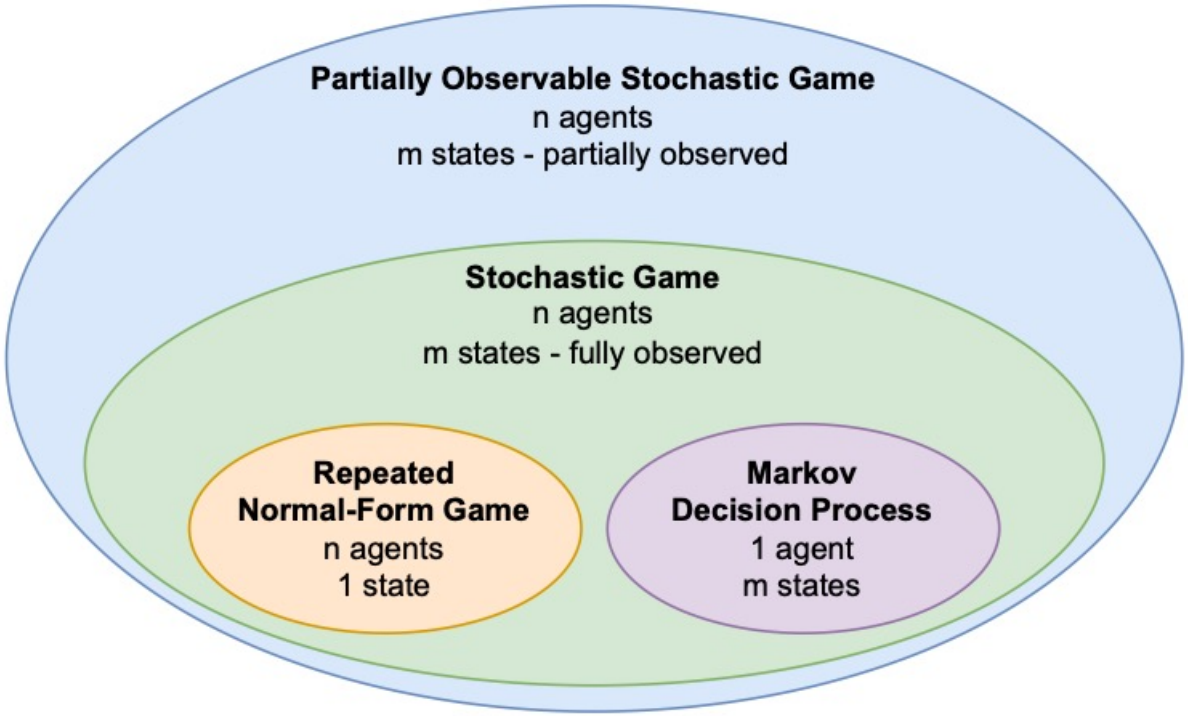


- At time step *t* all agents perform *collect*, each receiving reward +1
- Whose actions led to the reward?
- The agent on the left did not contribute
- Learning agents must disentangle the contributions of actions!

MARKOV MODELS

	No agents	Single agent	Multiple agents
State known	Markov Chain	Markov Decision Process (MDP)	Markov Games (a.k.a. Stochastic Games)
State observed indirectly	Hidden Markov Model (HMM)	Partially Observed Markov Decision Process (POMDP)	Partially-observable Stochastic Games (POSG)

HIERARCHY OF GAMES



NORMAL FORM-GAMES

Normal-form games define a **single** interaction between two or more agents, providing a simple kernel for more general games to build upon.

Normal-form games are defined as a 3 tuple $(I, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I})$:

- I is a finite set of agents $I = \{1, \dots, n\}$
- For each agent $i \in I$:
  - A_i is a finite set of actions
  - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : A \to \mathbb{R}$ where $A = A_1 \times \cdots \times A_n$ (set of **joint actions**).

NORMAL-FORM GAME PROCESS

In a normal-form game, there are **no time steps or states**. Agents choose an action and observe a reward.

The game proceeds as follows:

1. Each agent samples an action $a_i \in A_i$ with probability $\pi_i(a_i)$
2. The resulting actions from all agents form a **joint action**, $a = (a_1, \ldots, a_n)$
3. Each agent receives a reward based on its **individual** reward function and the **joint action**, $r_i = \mathcal{R}_i(a)$

CLASSES OF GAMES

Games can be classified based on the relationship between the agents' reward functions.

- In **zero-sum games**, the sum of the agents' reward is always 0
  i.e. $\sum_{i \in I} \mathcal{R}_i(a) = 0, \forall a \in A$
- In **common-reward** games, all agents receive the same reward ($R_i = R_j; \forall i, j \in I$)
- In **general-sum** games, there are no restrictions on the relationship between reward functions.

MATRIX GAMES

Normal-from games with **2** agents are also called **matrix games** because they can be represented using reward matrices.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock-Paper-Scissors

zero-sum

	A	B
A	10	0
B	0	10

Coordination Game

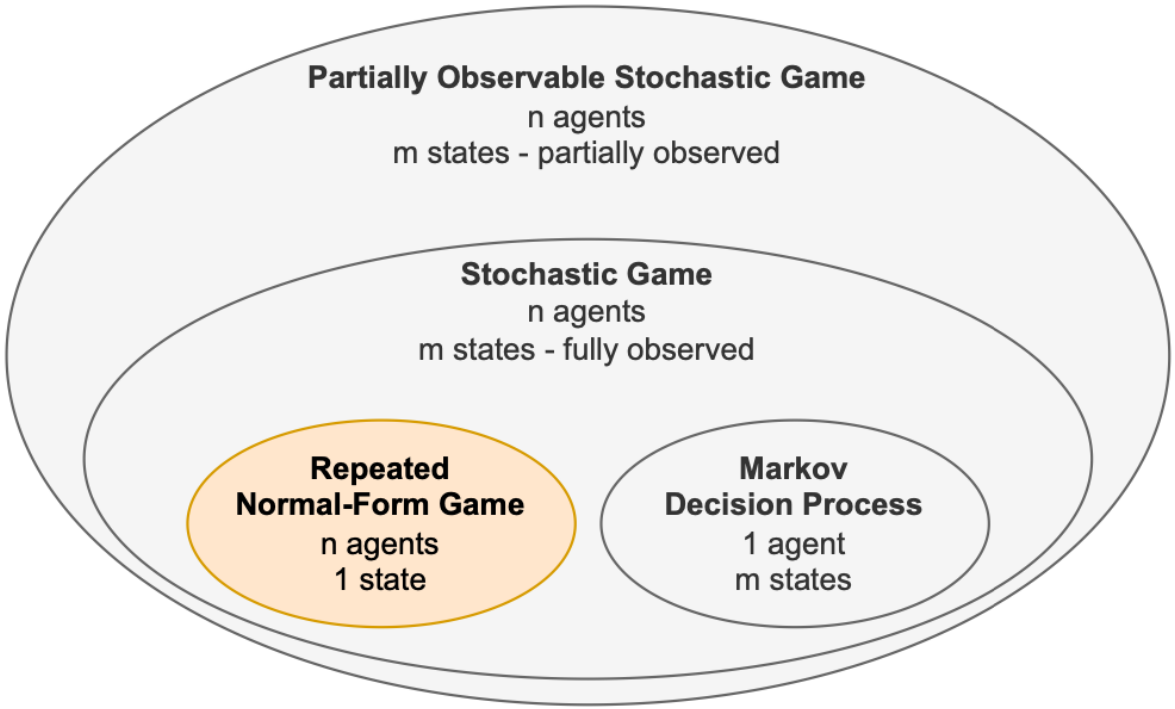
common-reward

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoner's Dilemma

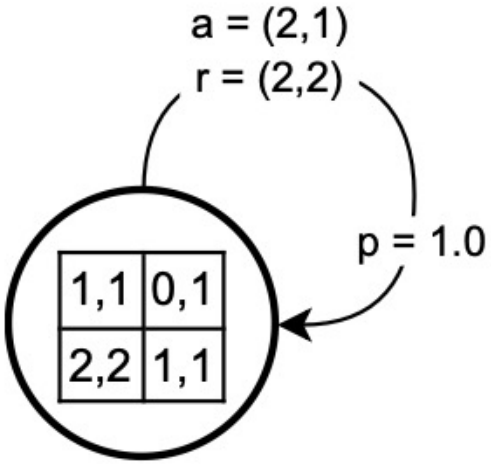
general sum

REPEATED NORMAL-FORM GAME

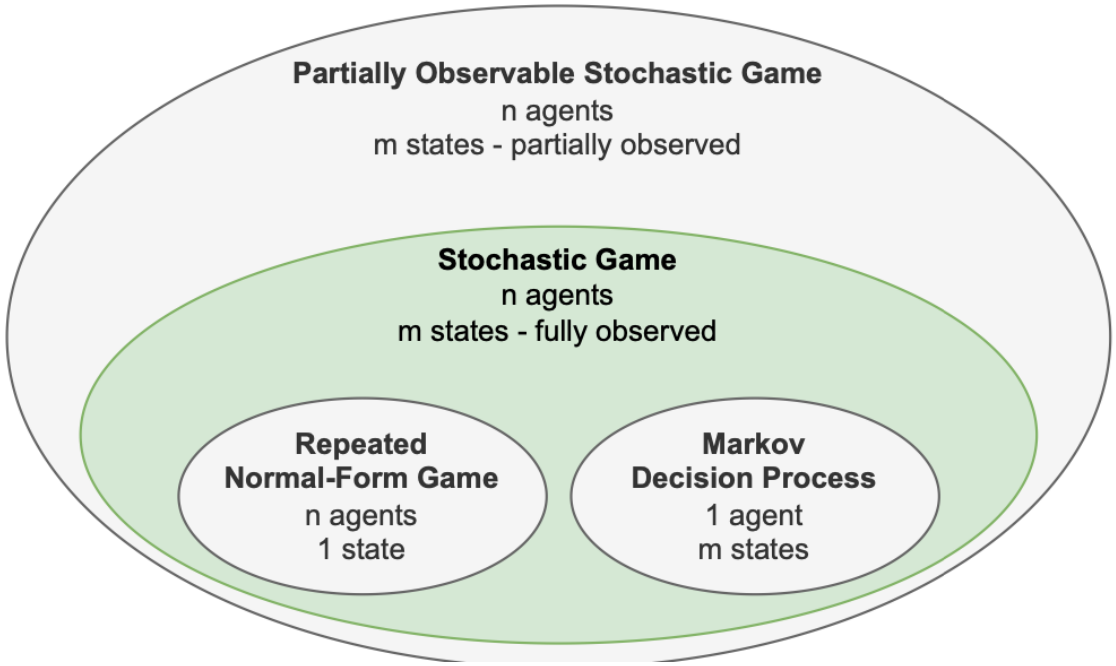


REPEATED NORMAL-FORM GAMES

To extend normal-form games to **sequential** multi-agent interaction, we can repeat the same game over T timesteps.



- At each time step t an agent i samples an action a_i^t
- The policy is now conditioned on a **joint-action** history $\pi_i(a_i^t|h^t)$ where $h^t = (a^o, \dots, a^{t-1})$
- In special cases, h^t contains n last joint actions. E.g. in a tit-for-tat strategy (Axelrod and Hamilton 1981), the policy is conditioned on a^{t-1}

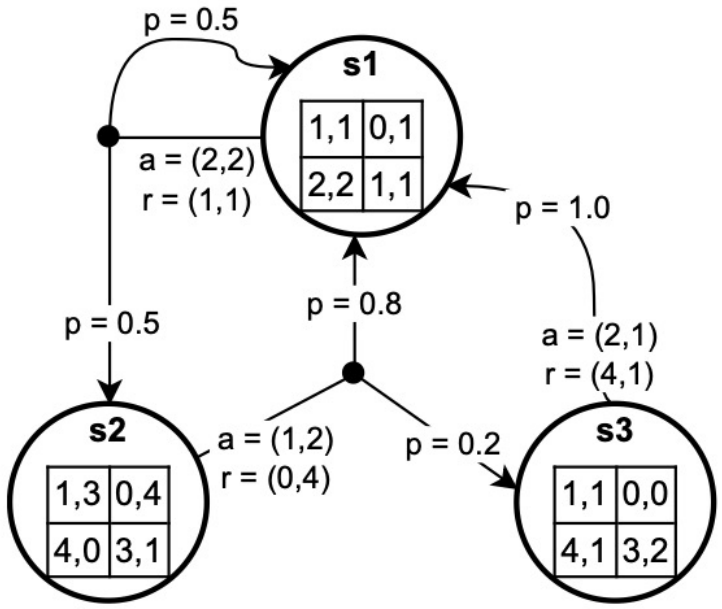

Partially Observable Stochastic Game
n agents
m states - partially observed
Stochastic Game
n agents
m states - fully observed
Repeated
Normal-Form Game
n agents
1 state
Markov
Decision Process
1 agent
m states

STOCHASTIC GAMES

Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

- I is a finite set of agents
- S is a finite set of states with subset of terminal states $\bar{S} \subset S$
- For each agent $i \in I$:
  - A_i is a set finite set of actions
  - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : S \times A \times S \to \mathbb{R}$ where A is the set of **joint** actions $A = A_1 \times \cdots \times A_n$
- $\mathcal{T}$ is the state transition function $\mathcal{T} : S \times A \times S \to [0,1]$
- μ is the initial state distribution $\mu : S \to [0, 1]$

STOCHASTIC GAMES

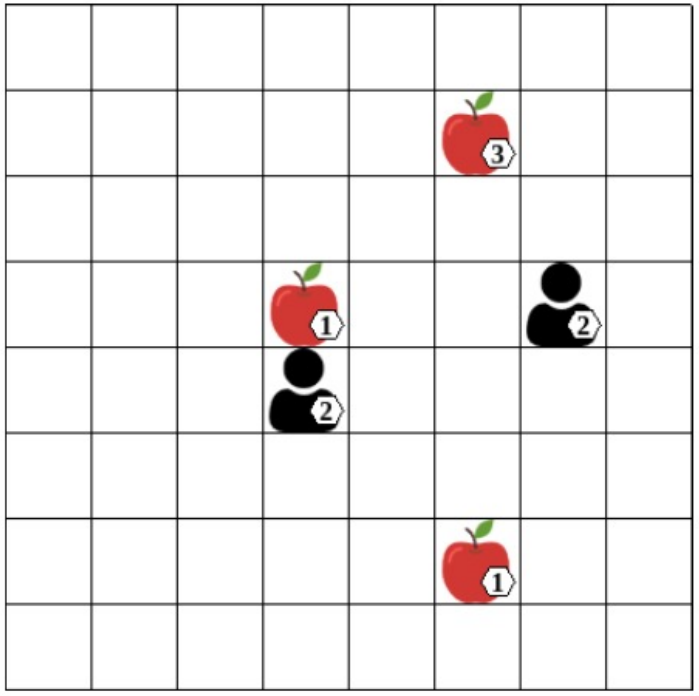


- Each **state** can be viewed as a **non-repeated normal-form game**
- Stochastic games can also be classified into: zero-sum, common-reward or general-sum
- The figure on the left shows a general-sum case

STOCHASTIC PROCESS

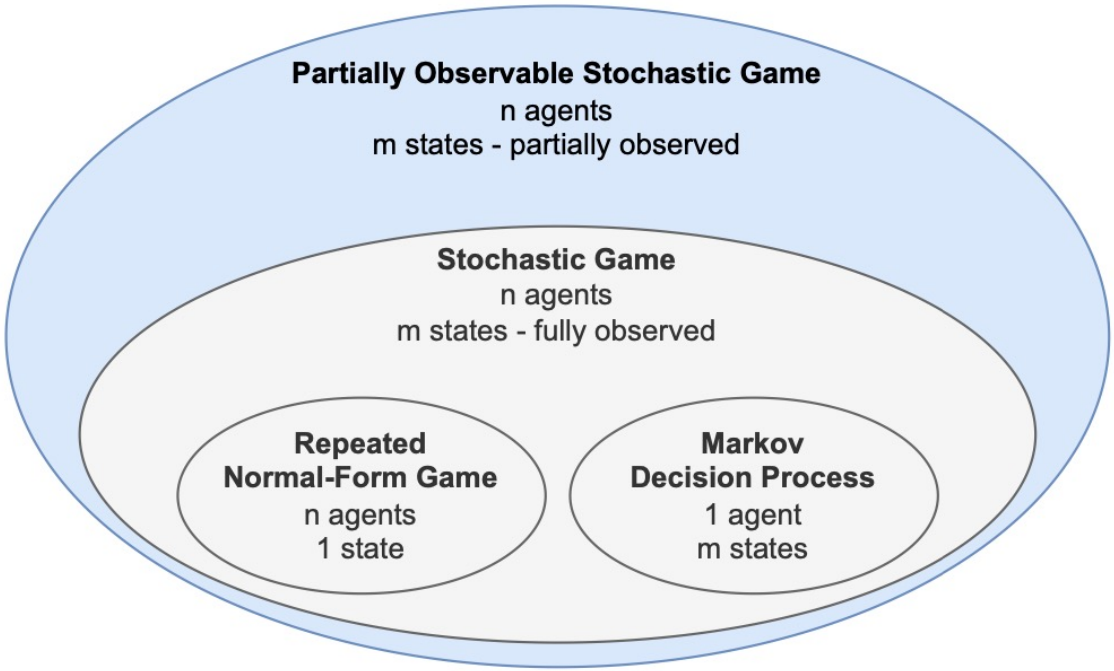
1. Initial state $s_0 \in S$ is samples from μ.
2. At time t each agent $i \in I$ observes the current state $st \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t|h^t)$
   - $h^t = (s^0, a^0, s^1, a^1, \ldots\ldots, s^t)$ is the **state-action history** containing the current state s^t and past **joint** actions and states
3. Given s^t and a^t, the game transitions into the next state $s^{t+1} \in S$ with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$
4. Each agent receives a reward according to their reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$
5. These steps are repeated until a terminal state $s^t \in \bar{S}$ is reached or a maximum number of T time steps is completed

EXAMPLE: LEVEL-BASED FORAGING



- $s \in S$: vector of x-y positions for agents/items, binary collection flags, levels for agents/items
- $a_i \in A_i$: move in four directions, collect item, or no operation (noop)
- $\mathcal{T}$: joint actions update state, e.g., two agents collecting an item switch its flag
- $\mathcal{R}$:
  - common-reward: +1 reward for any item collected by any agent
  - general-sum: +1 reward only for agents directly involved in item collection

PARTIALLY OBSERVABLE STOCHASTIC GAMES (POSG)



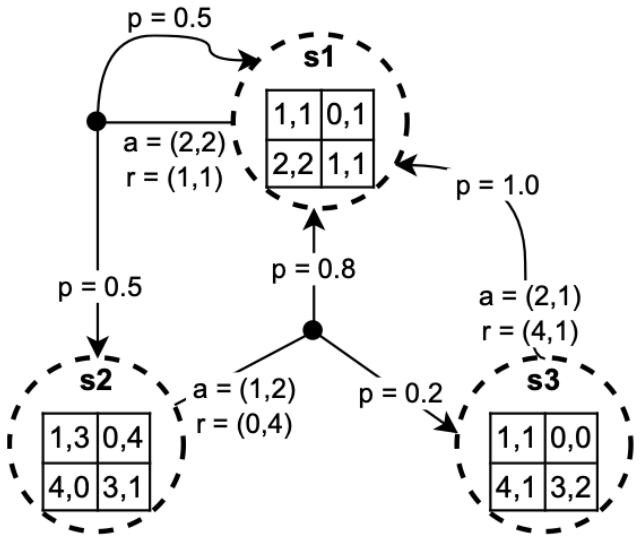
PARTIALLY OBSERVABLE STOCHASTIC GAMES (POSG)

At the top of the game model hierarchy, the most **general** model is the POSG

- POSGs represent complex decision processes with **incomplete information**
- Unlike in stochastic games, agents receive **observations** providing **incomplete information** about the state and agents' actions
- POSGs apply to scenarios where agents have limited sensing capabilities
  ⇒ e.g. autonomous driving
  ⇒ e.g. strategic games (e.g. card games) with private, hidden information

POSG DEFINITION

POSG is defined in the same way stochastic games are, with two additions. Thus, it is defined as an 8 tuple $(I, S, \{A_i\}_{i\in I}, \{\mathcal{R}_i\}_{i\in I}, \mathcal{T}, \mu, \{O_i\}_{i\in I}, \{\mathcal{O}_i\}_{i\in I})$



For each agent i we additionally define:

- a finite set of observation O_i
- an observation function $\{\mathcal{O}_i\}_{i\in I}$ such that $\mathcal{O}_i: A\times S\times O_i \to [0,1]$ and $\forall a \in A, s \in S: \sum_{o\in O_i} \mathcal{O}_i(a, s, o_i) = 1$

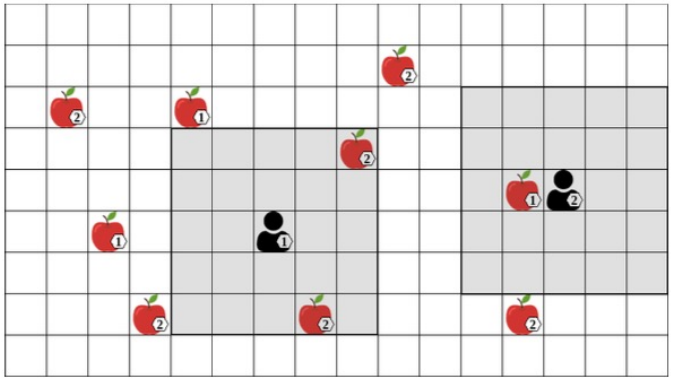
POSG PROCESS

1. Initial state s_0 sampled from $\mu(s)$
2. At each t every agent i receives an **observation** $o_i^t \in O_i$, with probability given by the observation function $\mathcal{O}_i(o^t|s^t, a^{t-1})$
3. Each agent then chooses an action a_i^t based on its policy π_i, which is conditioned on the agent's observation history, $h_i = (o_i^0, \dots, o_i^t)$.
   All agents' actions give the joint action $a^t = (a_1^t, \dots, a_n^t)$
4. Given the joint action, the environment transitions into the next state s^{t+1} with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$ and each agent receives a reward based on its reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$
5. This is done until a terminating state $s^t \in \bar{S}$ is reached or a maximum number of time steps is completed

THE OBSERVATION FUNCTION

POSG can represent diverse observability conditions. For example:

- modelling noise by adding uncertainty in the possible observation
- to limit the visibility region of agents (see LBF example)

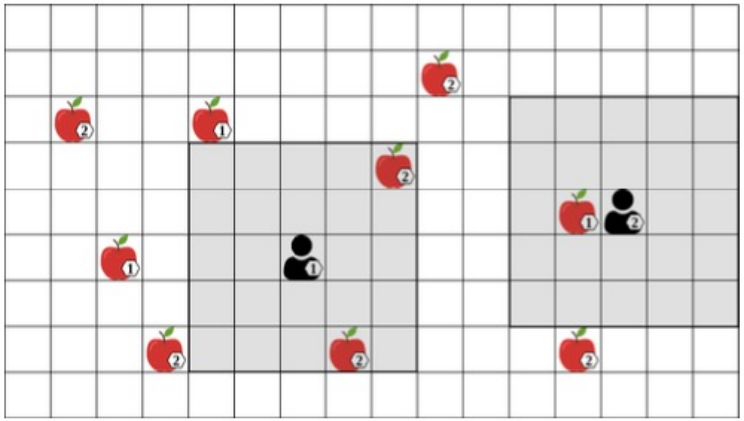


- Here, the agent can only see some parts of the state and joint action
- $o_i^t = (\bar{s}^t, \bar{a}^t)$ where $\bar{s}^t \subset s^t, \bar{a}^t \subset a^t$

BELIEFE STATE

In partially observable settings, it becomes more challenging to infer optimal actions. For example:

- Optimal action for agent 1 is to move left towards level 1 apple
- But level 1 apple is not directly observable
- Agent 1 can hold a **belief state** b_i^t , a probability distribution over possible state $s \in S$
- Agent 1 might have seen the level-1 apple previously and can thus 'remember' its location



SINGLE AGENT BELIEF UPDATE

To simplify, let's consider the single-agent perspective:

- The initial belief state is given by $b_i^0 = \mu$
- After taking action a^t and observing o^{t+1}, the belief state b^t is updated to b_i^{t+1} using a Bayesian update:

$$b_i^{t+1}(s') \propto \sum_{s \in S} b_i^t(s) \mathcal{T}\left(s' \middle| s, a_i^t\right) \mathcal{O}_i(o_i^{t+1} | a_i^t, s')$$

In MARL this type of update is typically **infeasible**:

- High-dimensional state spaces make storage and updates of beliefs intractable
- In MARL for POSG, agents assumed not to know $(S, \mathcal{T}, \mathcal{O}_i)$
- Deep learning can be used to approximate state information

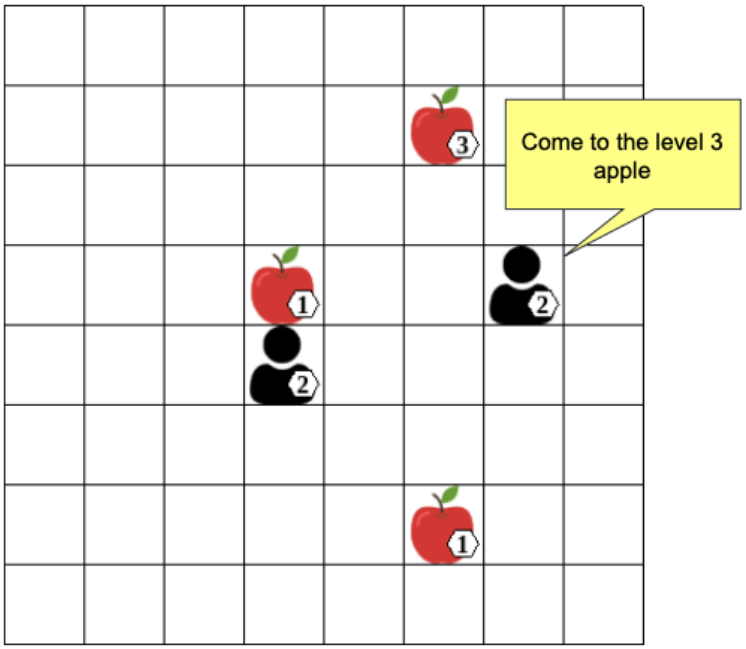
COMMUNICATION

AARHUS
UNIVERSITY
DEPARTMENT OF ENGINEERING

AUTUMN 2025

LUKAS ESTERLE
ASSOCIATE PROFESSOR

MODELING COMMUNICATION



- Using games, we can model more complex agent interactions, such as communication
- We can view communication as a type of action that other agents can observe without affecting the state of the environment
- Agents learn communication meanings through trials and observations, identical to environment actions
- This can lead to the evolution of a shared language or protocol

COMMUNICATION ACTIONS

To model communication, we can extend the action set of agents:

$$A_i = X_i \times M_i$$

- Where M_i is a set of possible messages $\{m_1, m_2, m_3, \dots\}$ and X_i is the set of environment actions
- The action a_i can thus be expressed as $(x_i, m_i) \in A_i$

COMMUNICATION IN STOCHASTIC GAMES

- Agents observe the current state s^t and previous joint action a^{t-1}
- Communication action m_i^{t-1} by agent i is part of a^{t-1} and observed by all agents
- State transitions are independent of the joint communication actions $M = \times_{i \in I} M_i$

$$\forall s, s' \in S \forall a \in A, m \in M : T(s'|s,a) = T(s'|s, \langle (a_1, m_1), \ldots, (a_n, m_n) \rangle)$$

COMMUNICATION IN POSG

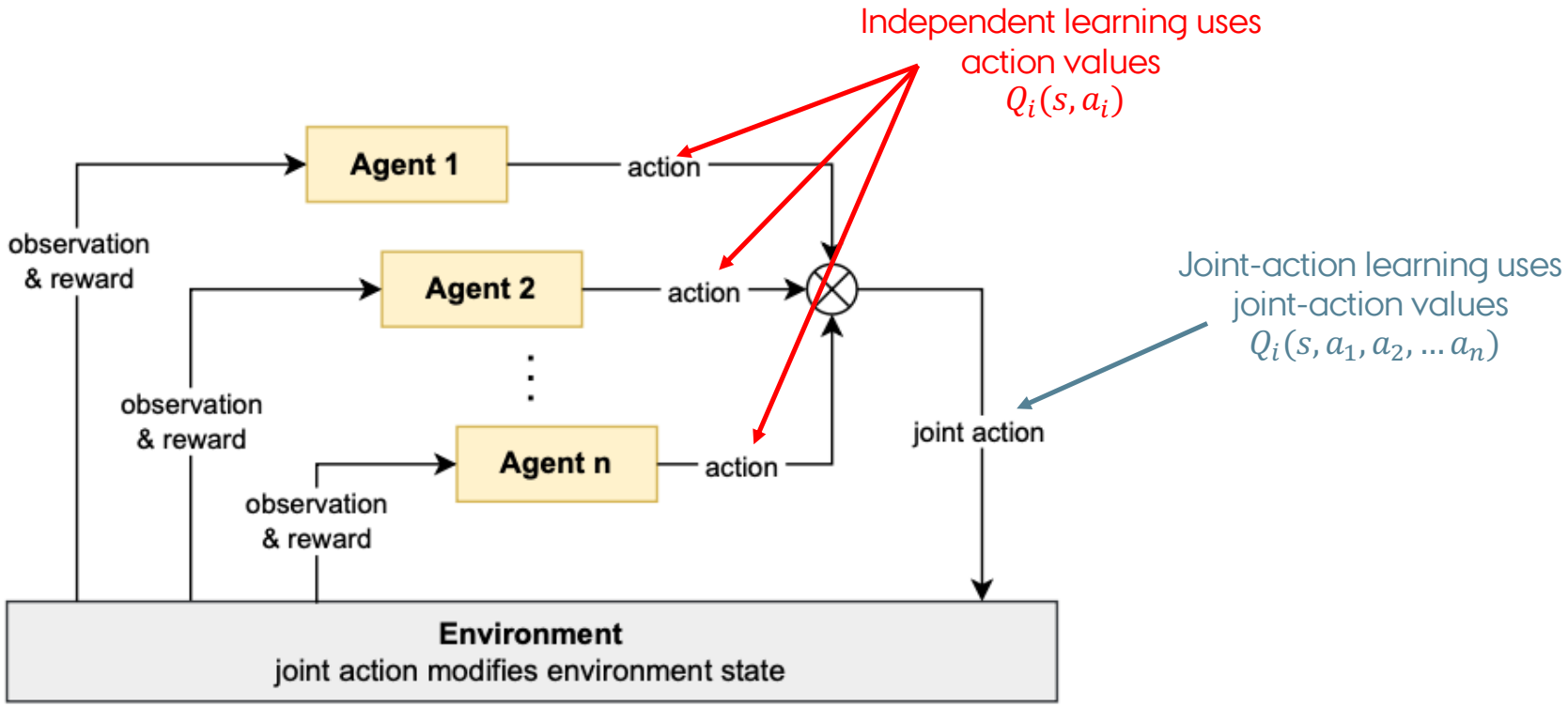
- In POSG we can use the observation function $\mathcal{O}_i$ to model noisy or unreliable communication
- We can define the observation as $o_j^t = [\ \bar{s}^t, w_1^{t-1}, \ldots, w_n^{t-1}]$
  - $\bar{s}^t$ is some partial information about the state
  - w_j^{t-1} is a message from the agent j at time step $t-1$ which has been augmented by $\mathcal{O}_i$
  - E.g. $w^{t-1} = f(m_j^{t-1})$ where $f(m_j^{t-1}) = m_j^{t-1} + \eta$, and η is some random noise component.
- You could also model $\mathcal{O}_i$ to hide messages such that $w_1^{t-j} = \emptyset$ if agent i is too far from agent j

GAME THEORY ASSUMPTION

- In game theory, we typically assume that all agents know all components of the game **(complete knowledge games)**
- Agents know all agents' **action spaces and reward functions**
- Knowledge of other agents' reward functions may be used for informing the agent's **best response** action
- Knowledge of the transition function (T) allows for predicting state changes and **planning** actions multiple steps ahead

MARL ASSUMPTIONS

- In MARL, we assume **limited knowledge**, i.e. no knowledge of transition function $\mathcal{T}$ and no knowledge of agents' reward functions $\mathcal{R}_i$
- Additional assumption can be added, and specific knowledge of the game can be held **mutually** or **asymmetrically**
- We usually assume the **number of agents to be fixed**, although recent research has looked at *open* multi-agent systems, this will not be covered in these lectures


Independent learning uses
action values
$Q_i(s, a_i)$
Agent 1
action
observation
& reward
Agent 2
action
observation
& reward
Agent n
action
observation
& reward
joint action
Joint-action learning uses
joint-action values
$Q_i(s, a_1, a_2, \ldots a_n)$
Environment
joint action modifies environment state

PROBLEM OF JOINT-ACTION LEARNING

$Q_i(s, a_1, \dots, a_n)$ is not enough to find optimal action for agent i

- Cannot evaluate $\max_{a_i} Q_i(s, a_1, \dots, a_n)$

  ⇒ Optimal action depends on actions of other agents!

We have to define:

1. How to select action from Q_i?
2. How to update Q_i?

IDEA

Joint-action value functions define a normal-form game:

- Each agent stores Q-function Q_j for every agent $j \in I$
  (assumes agents can observe all agents' actions and rewards)
- Define reward functions for normal-form game in state s as

$$\mathcal{R}_j(a_1, \dots, a_n) = Q_j(s, a_1, \dots, a_n)$$

- We can *solve* the normal-form game defined by

$$\Gamma_s = (\mathcal{R}_1 = Q_1(s,\cdot), \cdots, \mathcal{R}_n = Q_n(s,\cdot))$$

SOLUTION

Solution of Γ is a joint policy $\pi_s^* = (\pi_{s,1}^*, \dots, \pi_{s,n}^*)$ with certain properties

- e.g. compute minimax solution or Nash equilibrium of Γ_s
  ⇒ Use $\pi_{s,i}^*$ to select action for agent i

Value of Γ_s for agent j is its expected reward under joint policy π_s^*

$$Value_j(\Gamma_s) = \sum_{a \in A} Q_j(s,a)\pi_s^*(a)$$

⇒ Update Q_j towards update target: $r_j + \gamma\, Value_j(\Gamma_{s'})$

MINIMAX

Minimax is a solution concept defined for **two-agent zero-sum** games. Recall that one agent's reward is the negative of the other agent's reward.

- minimizing the possible loss for a worst case (maximum loss) scenario
- The existence of minimax solution in normal-form games was first proven by John von Neumann (1928)
- Minimax solutions also exist in two-agent zero-sum stochastic games with finite episode lengths, like chess and Go.

In a two-agent zero-sum game, a joint policy $\pi = (\pi_i, \pi_j)$ is minimax solution if

$$
\begin{aligned}
U_i(\pi) &= \max_{\pi_i} \min_{\pi_j} U_i(\pi_i', \pi_j') \\
&= \min_{\pi_j} \max_{\pi_i} U_i(\pi_i', \pi_j') \\
&= -U_j(\pi)
\end{aligned}
$$

NASH EQUILIBRIUM

Nash equilibrium solution concept applies the idea of **a mutual best response** to general-sum games with two or more agents.

- No agent i can improve its expected returns by changing its policy π_i assuming other agents policies remain fixed:

$$\forall i, \pi_i' : \; U_i(\pi_i', \pi_{-i}) \; \leq \; U_i(\pi)$$

- Each agent's policy is a **best response** to all other agent's policies
- John Nash (1950) proved the existence of such a solution for **general-sum non-repeated normal-from games**

JOINT-ACTION LEARNING PSEUDOCODE

Algorithm Joint-action learning with game theory (JAL-GT)

// Algorithm controls agent i

Initialize: $Q_j(s,a) = 0$ for all $j \in I$ and $s \in S, a \in A$
Repeat for every episode:
for $t = 0, 1, 2, \ldots$ **do**
    Observe current state s^t
    With probability ϵ: choose random action a_i^t
    Otherwise: solve Γ_{s^t} to get policies $(\pi_1, \ldots, \pi_n)$, then sample action $a_i^t \sim \pi_i$
    Observe joint action $a^t = (a_1^t, \ldots, a_n^t)$, rewards $r_1^t, \ldots, r_n^t$, next state s^{t+1}
    for all $j \in I$ **do**
        $Q_j(s^t, a^t) \leftarrow Q_j(s^t, a^t) + \alpha \left[r_j^t + \gamma Value_j(\Gamma_{s^{t+1}}) - Q_j(s^t, a^t) \right]$

MINIMAX-Q, NASH-Q CE-Q

Minimax-Q solves Γ_s via minimax solution (Littman, 1994)

- Converges to unique minimax values in two-agent zero-sum stochastic games
- Minimax profile can be computed with linear programming (LP)

Nash-Q solves Γ_s via Nash equilibrium (Hu and Wellman, 2003)

CE-Q solves Γ_s via correlated equilibrium (Greenwald and Hall, 2003)

- Converges to equilibrium under highly restrictive conditions
  ⇒ Problem: often no unique equilibrium value in general-sum games
- Compute CE with LP, compute NE with quadratic programming

AGENT MODELLING & BEST RESPONSE

Game theory solutions are normative: they *prescribe* how agents *should* behave

- e.g. minimax assumes worst-case opponent

What if agents don't behave as prescribed by solution?

- e.g. minimax-Q was unable to exploit hand-built opponent in soccer example

Other approach: agent modelling with best response

- Learn models of other agents to predict their actions
- Compute optimal action (best response) against agent models

POLICY RECONSTRUCTION & BEST RESPONSE

Policy reconstruction: learn model $\hat{\pi}_j \approx \pi_j$ from past observed actions

In general, can train model with supervised learning on data $\{(s^\tau, a^\tau)\}_{\tau=1}^{t-1}$

- E.g. look-up table, neural network, finite state machine, …
- Model should support incremental updating

Given models for other agents $\hat{\pi}_{-i} = \{\hat{\pi}_j\}_{j \neq i}$, compute best response $\pi_i \in BR_i(\hat{\pi}_{-i})$

FICTITIOUS PLAY

Fictitious play (Brown 1951) algorithm for non-repeated normal-form games Each agent *i* models other agents *j* as stationary distribution:

$$\hat{\pi}_j(a_j) = \frac{C(a_j)}{\sum_{a'_j \in A_j} C(a'_j)}$$

$C(a_j)$ is number of times agent j chose action a_j in prior episodes

In each episode, agents choose best-response action:

$$BR_i(\hat{\pi}_{-i}) = \arg\max_{a_i \in A_i} \sum_{a_{-i} \in A_{-i}} \mathcal{R}_i(\langle a_i, a_{-i} \rangle) \prod_{i \neq j} \hat{\pi}_j\ (a_j)$$

JOINT-ACTION LEARNING WITH AGENT MODELLING

Extend fictitious play approach to stochastic games by using joint-action learning with agent models

Agents models other agents j, this time conditioned on states s:

$$\hat{\pi}_j(a_j \mid s) = \frac{C(s, a_j)}{\sum_{a'_j \in A_j} C(s, c'_j)}$$

Given models $\{\hat{\pi}_j\}_{j \neq i}$, action values are defined as:

$$AV_i(s, a_i) = \sum_{a_{-i} \in A_{-i}} Q_i(s, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \hat{\pi}_j(a_j \mid s)$$

⇒ Use AV_i to select optimal actions and as learning update targets

JOINT-ACTION LEARNING WITH AGENT MODELLING - PSEUDOCODE

Algorithm Joint-action learning with agent modeling (JAL-AM)

Initialize:
  $Q_i(s, a) = 0$ for all $s \in S, a \in A$
  Agent models $\hat{\pi}_j(a_j \mid s) = \frac{1}{|A_j|}$ for all $j \neq i, a_j \in A_j, s \in S$
Repeat for every episode:
for $t = 0, 1, 2, \ldots$ **do**
    Observe current state s^t
    With probability ϵ: choose random action a_i^t
    Otherwise: choose best-response action $a_i^t \in \arg\max_{a_i} AV_i(s^t, a_i)$
    Observe joint action $a^t = (a_1^t, \ldots, a_n^t)$, reward r_i^t, next state s^{t+1}
    Update agent models $\hat{\pi}_j$ with new observations (e.g., (s^t, a_j^t))
    $Q_i(s^t, a^t) \leftarrow Q_i(s^t, a^t) + \alpha \left[r_i^t + \gamma \max_{a_i'} AV_i(s^{t+1}, a_i') - Q_i(s^t, a^t) \right]$

POLICY-BASED LEARNING

All algorithms so far derive policies from learned action-value functions

- Has important limitations, e.g. algorithms using best-response actions from values (e.g. fictitious play, JAL-AM) cannot represent probabilistic equilibria
- Fictitious play is unable to represent uniform-random NE in Rock-Paper-Scissors

Policy-based learning instead uses learning data to directly optimise policies

- Use *parameterised policies* that are differentiable
- Use *gradient-ascent techniques* to optimise parameters
  ⇒ Can directly learn action probabilities in policies!

GRADIENT ASCENT IN EXPECTED REWARD

Gradient-ascent learning in non-repeated normal-form games with two agents i, j and two actions

Reward matrices:

$$\mathcal{R}_i = \begin{bmatrix} r_{1,1} & r_{1,2} \\ r_{2,1} & r_{2,2} \end{bmatrix} \qquad \mathcal{R}_j = \begin{bmatrix} c_{1,1} & c_{1,2} \\ c_{2,1} & c_{2,2} \end{bmatrix}$$

Policies with parameters $\alpha, \beta \in [0, 1]$:

$$\pi_i = (\alpha, 1 - \alpha) \qquad \pi_j = (\beta, 1 - \beta)$$

GRADIENT ASCENT IN EXPECTED REWARD

Update policy in direction of gradient in expected reward using step size $\kappa > 0$:

$$\alpha^{k+1} = \alpha^{k} + \kappa \frac{\partial U_i(\alpha^{k}, \beta^{k})}{\partial \alpha^{k}}$$

$$\beta^{k+1} = \beta^{k} + \kappa \frac{\partial U_j(\alpha^{k}, \beta^{k})}{\partial \beta^{k}}$$

Partial derivative of an agent's expected reward with respect to its policy:

$$\frac{\partial U_i(\alpha, \beta)}{\partial \alpha} = \beta u + (r_{1,2} - r_{2,2})$$

$$\frac{\partial U_j(\alpha, \beta)}{\partial \beta} = \alpha u' + (c2{,}1 - c2{,}2).$$

Where $u = r_{1,1} - r_{1,2} - r_{2,1} + r_{2,2}$ and $u' = c_{1,1} - c_{1,2} - c_{2,1} + c_{2,2}$

LEARNING DYNAMICS OF INFINITESIMAL GRADIENT ASCENT

What joint policy will agents converge to?
⇒ Can analyse learning dynamics via dynamical systems theory!

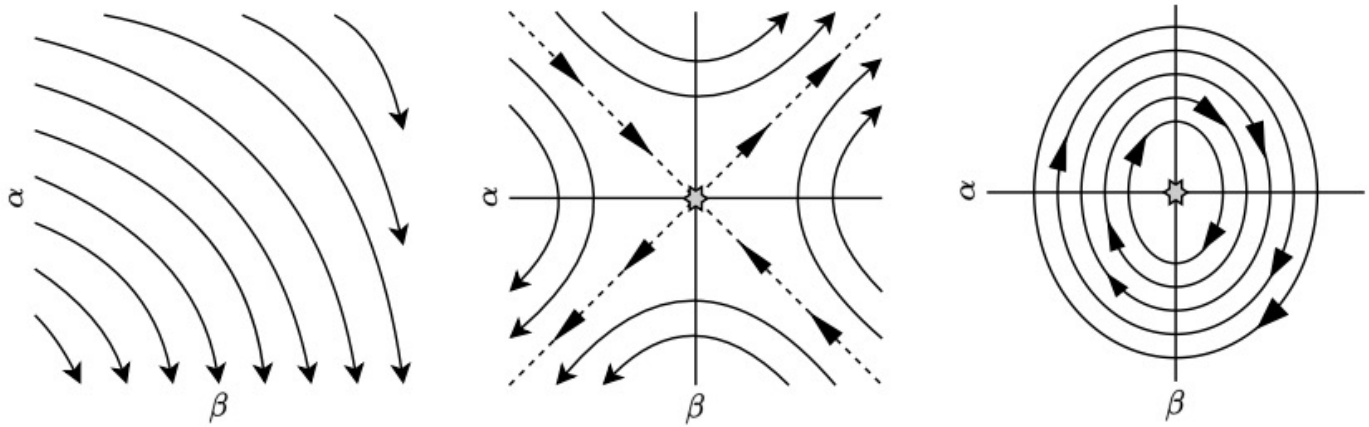
Infinitesimal Gradient ascent (IGA): use infinitesimal step size $\kappa \to \infty$

Joint policy given by $(\alpha(t), \beta(t))$ will follow continuous trajectory according to differential equation:

$$\begin{bmatrix} \frac{\partial \alpha}{\partial t} \\ \frac{\partial \beta}{\partial t} \end{bmatrix} = \begin{bmatrix} 0 & u \\ u' & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} + \begin{bmatrix} (r_{1,2} - r_{2,2}) \\ (c_{2,1} - c_{2,2}) \end{bmatrix}$$

LEARNING DYNAMICS OF INFINITESIMAL GRADIENT ASCENT

Learning dynamics of (α, β) will follow one of three types of trajectories:



(a) F not invertible

(b) F has purely real eigenvalues

(c) F has purely imaginary eigenvalues

IGA CONVERGENCE

- (α, β) does not converge in all cases
- If (α, β) does not converge, then average rewards during learning converge to expected rewards of some NE
- If (α, β) converges, then converged joint policy is a NE

WIN OR LEARN FAST (WOLF)

By using a variable step size κ, we can ensure that IGA policies *always* converge to NE

Win or Learn Fast (WoLF): (Bowling and Veloso 2002)

- learn fast (use larger κ) when "losing"
- learn slow (use smaller κ) when "winning"

Winning/losing depends on current expected reward compared to NE rewards

NO-REGRET LEARNING

JAL-GT and JAL-AM algorithms use solution concepts and agent modelling to learn joint policies

Now: no-regret learning algorithms that use regret definitions to learn policies

- We consider two simple regret matching algorithms (Hart and Mas-Colell, 2000)
- In normal-form games, their empirical action distributions converge to set of (coarse) correlated equilibrium

UNCONDITIONAL REGRET MATCHING

Unconditional regret matching: compute action probabilities proportional to (positive) average *unconditional* regrets of actions $a_i \in A_i$

$$Regret_i^z(a_i) = \sum_{e=1}^{Z} [\mathcal{R}_i(\langle a_i, a_{-i}^e \rangle - \mathcal{R}_{i(a^e)})]$$

a^e is joint action from past episodes $e = 1, \ldots, z$.

Each agent i starts with a random initial policy π_i^1, then update π_i^z to

$$\pi_i^{z+1}(a_i) = \frac{[\bar{R}_i^z(a_i)]_+}{\sum_{a_i' \in A_i} [\bar{R}_i^z(a_i')]} \quad \text{with } \bar{R}_i^z(a_i) = \frac{1}{z} Regret_i^z(a_i)$$

where $[x]_+ = \max[x, 0]$.

CONDITIONAL REGRET MATCHING

Conditional regret matching: compute action probabilities proportional to (positive) average *conditional* regrets with respect to most recent selected action a_i'

$$Regret_i^z\ (a_i', a_i) = \sum_{e: a_i^e = a_i'} [\mathcal{R}_i(\langle a_i, a_{-i}^e \rangle) - \mathcal{R}_i(a^e)]$$

Each agent i starts with a random initial policy π_i^1, then update π_i^z to

$$\pi_i^{z+1}(a_i) = \begin{cases} \frac{1}{\eta}[\bar{R}_i^z(a_i^z, a_i)]_+ & \text{if } a_i \neq a_i^z \\ 1 - \sum_{a_i' \neq a_i^z} \pi_i^{z+1}(a_i') & \text{otherwise} \end{cases}$$

with $\bar{R}_i^z(a_i', a_i) = \frac{1}{z} Regret_i^z(a_i', a_i)$

where $\eta > 2 \cdot \max_{a \in A} |\mathcal{R}_i(a)| \cdot (|A_i| - 1)$ is a parameter.

SUMMARY OF MARL

Games:

- Normal-form games → *n* agents, 1 state
- Stochastic games → *n* agents, *m* states – fully observable
- Partially observable games → *n* agents, *m* states – NOT fully observable

Communication enables agents to share their observations and actions

SUMMARY OF MARL

MARL aims to learn *joint-actions* instead of simply individual actions

- Solve as normal form game in each state
- Operate on best response and policy reconstruction
- Utilise fictitious play

Model the other agents at runtime – use gradient ascent

- Regret matching to adjust models
- Variable step size to ensure convergence

QUESTIONS (COORDINATION + PLANNING)

Group 1:

Compare different auction mechanisms, in terms of how they work, and their strategies.

Describe the planning problem, how do we formalize it, what is it, what is it not?

Group 2:

Compare different voting mechanisms. Reflect on how you could use them in a MAS.

What kind of methods can we use to solve the planning problem?

Group 3:

What algorithms can we use to select a leader? Describe them in short.

How would you combine coordination mechanisms and planners to (re-)plan in a MAS? Think of concrete scenarios.

Group 4:

What are the different ways agents can coordinate with each other? What are some negotiation strategies agents can use?

Think of scenarios where you would prefer planning to coordination and vice versa.